

COMP9001

Lecture 8

More Flow Control

Presenter: Dr Armin Chitizadeh

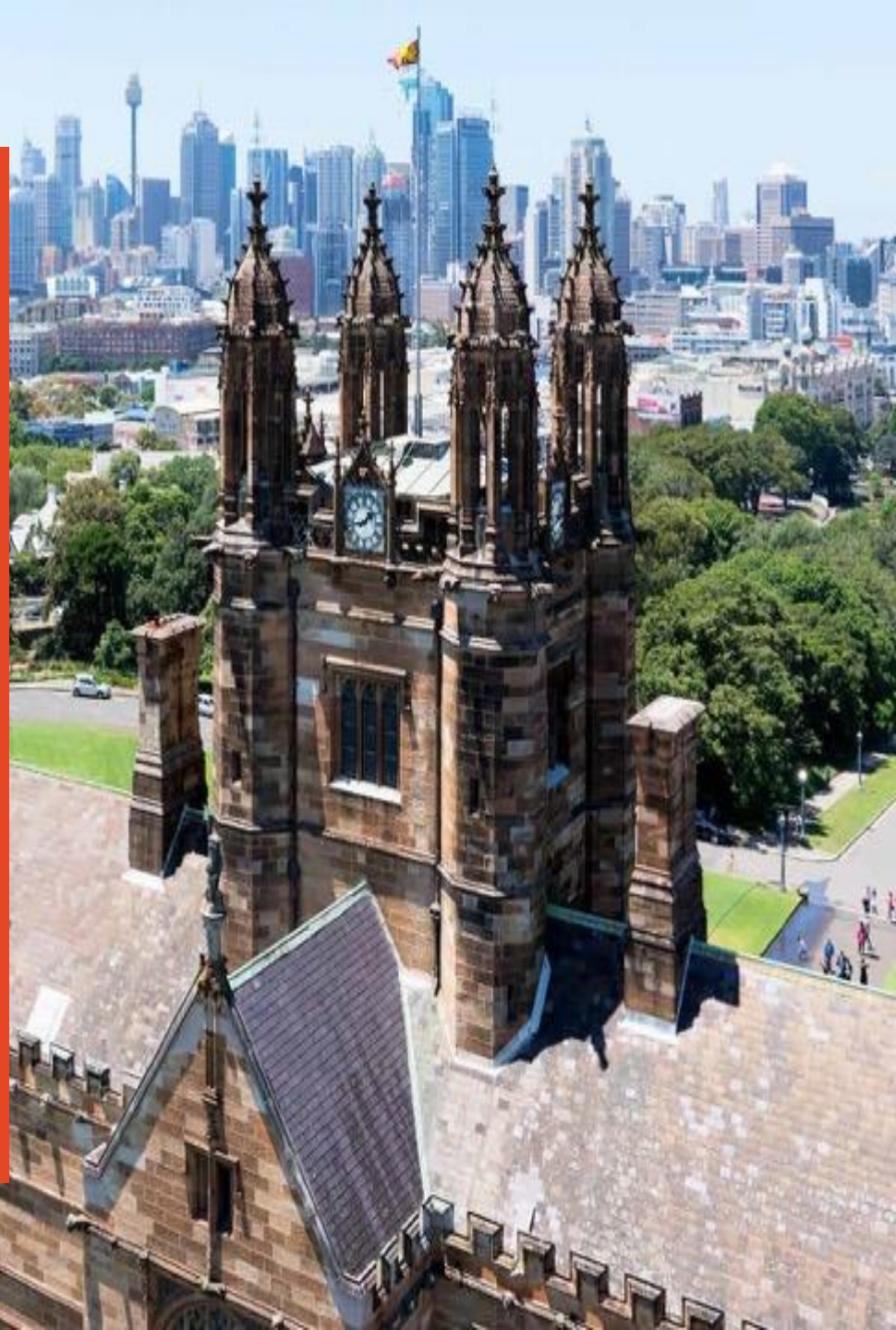
Version 1.0



THE UNIVERSITY OF
SYDNEY

Inspired by Dr Hazem El-Alfy
Dr Karlos Ishac 2025 slides

CRICOS 00026A TEO SA PRV12057



Students with tutorial on Monday have three options

- Attend other tutorials
- Attend the tutorial on zoom (Links to be shared on ed)
- Watch the tutorial recording (Links to be shared on ed)

Interviews for Assignment 1

1. The interview will be held on **MS Teams**.
2. The interview will be **recorded**. You need to have your **Camera ON** and **share your Desktop** to show your assignment.
3. When booking please make sure that your **Time Zone** is set to **Sydney Time**. You need to set it manually as follows:
 - Book **ONLY** one Time slot. If there are more than one time booked per a student, I will keep the latest one and cancel the rest.
 - Make sure you have access to **Mic** and **Camera** before your interview.
 - You need to use your **USYD email address**. (If it was not working, use a personal email address)
 - Show your **Student ID** when you enter the interview.
 - Duration of the interview is **10 min**.

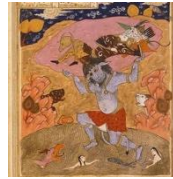
Where are we in the unit?



Week 1
Intro



Week 2
Variables



Week 3
Conditionals



Week 4
Containers



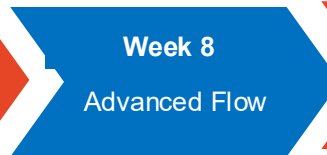
Week 5
Loops



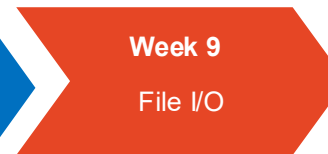
Week 6
Flow and
Functions



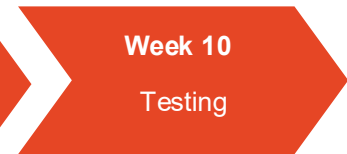
Week 7
Classes/Objects



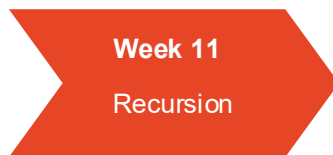
Week 8
Advanced Flow



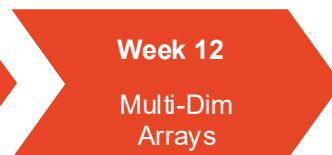
Week 9
File I/O



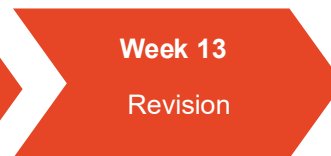
Week 10
Testing



Week 11
Recursion



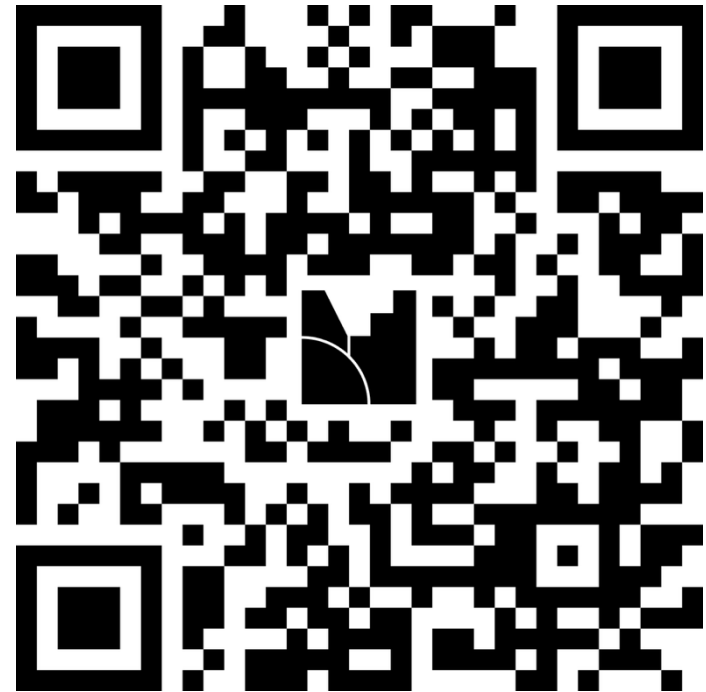
Week 12
Multi-Dim
Arrays



Week 13
Revision

What is one superstition that you believe in?

- <https://www.menti.com/alz81tvzhyzv>



For this week, you will learn how to be a better programmer

- Why didn't we cover these earlier?



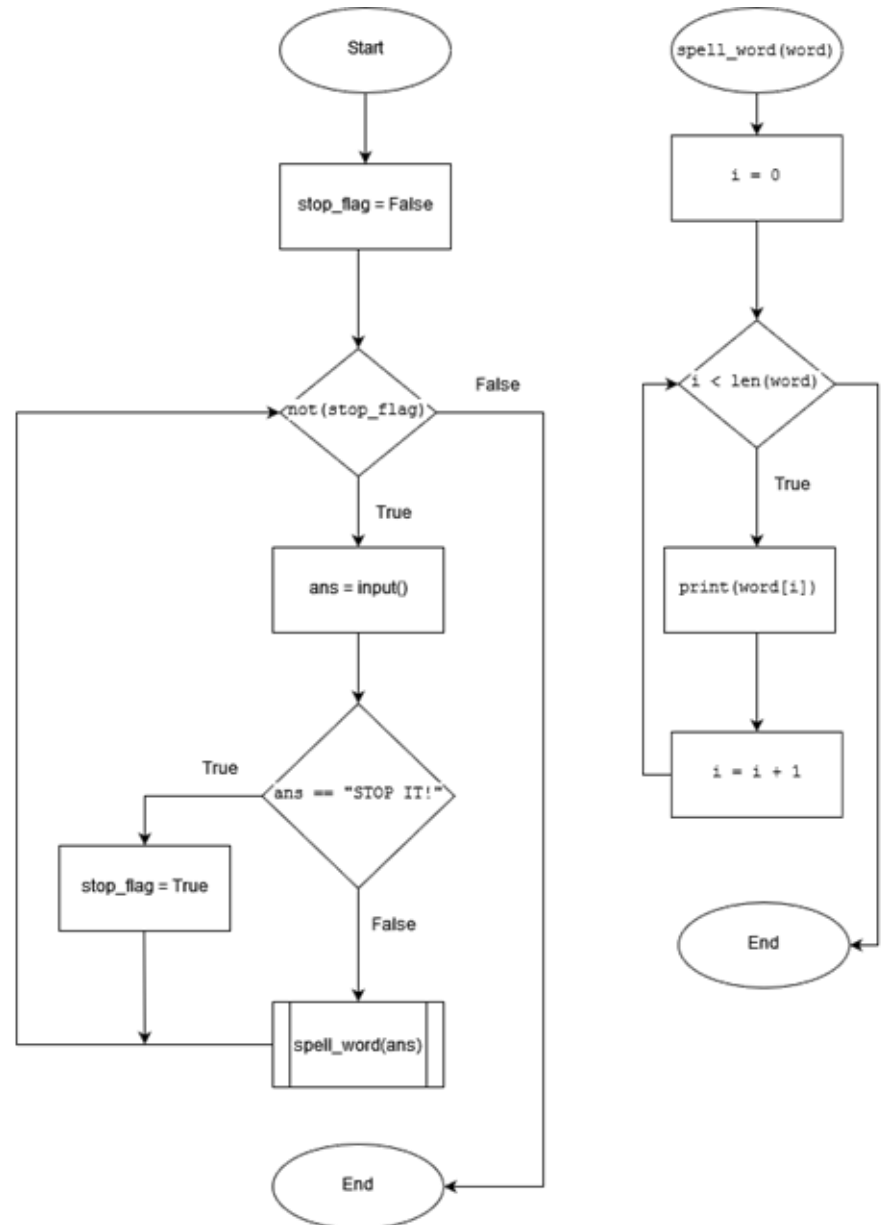
For this week, you will learn how to be a better programmer

- Why didn't we cover these earlier?



Flow of Execution – What we know so far?

- Runs sequentially from top to bottom except in cases:
 - **Functions**
 - Entire function body executes during a function call.
 - **Conditionals**
 - Execution path depends on whether a condition is True or False.
 - **Loops**
 - Actions are repeated depending on whether a condition is True (count-controlled and event-controlled).



Flow of Execution – What ifs?

- What if:
 - We want to **continue** a loop but stop for a while to take a **break**?
 - I want an early exit from a function; can I **return** now?
 - I know something can go wrong during execution when usage is different from design. Should I **raise** this issue with my boss?

This Week

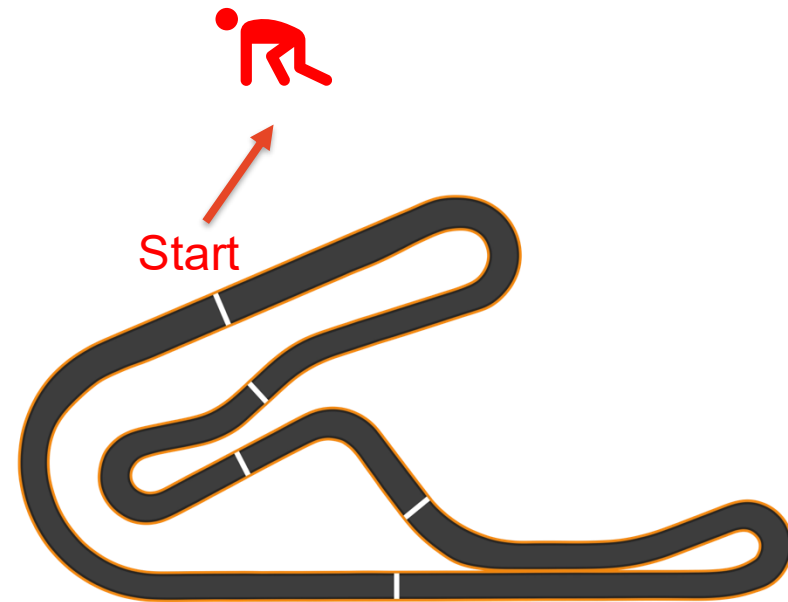
- Escape a loop early:
 - `break`
 - `continue`
- Functions re-visit: multiple exits/multiple values upon exit
 - `return`
- Stopping execution
 - `raise`
 - `assert`
 - Follow up exception handling: `try-except-finally` blocks.

Loops IRL

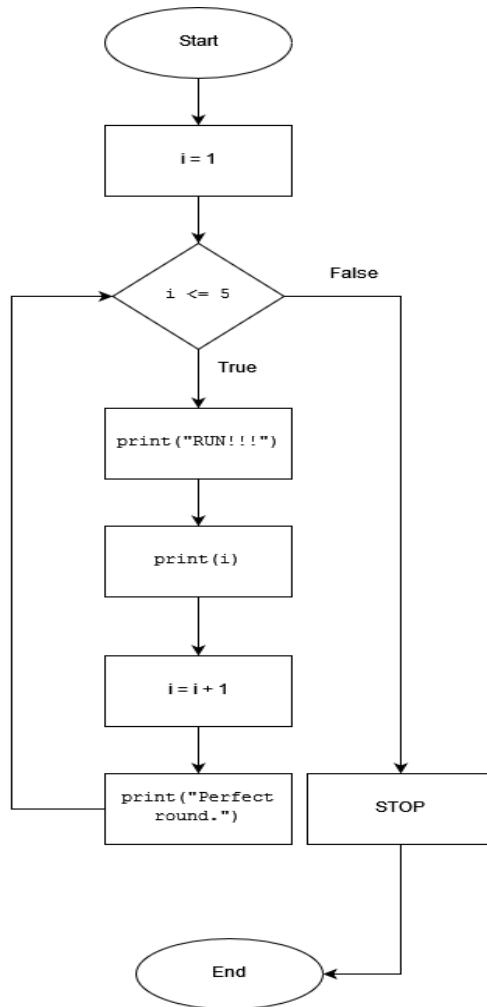
- A race requires participants to run around the track **FIVE(5)** times. That's you at the starting line!
- This is the Python program that your actions are based on:

```
1 i = 1
2 while i<=5:
3     print("RUN!!!")
4     print(i)
5     i = i + 1
6     print("Perfect round.")
7 print("STOP")
8 # End program
```

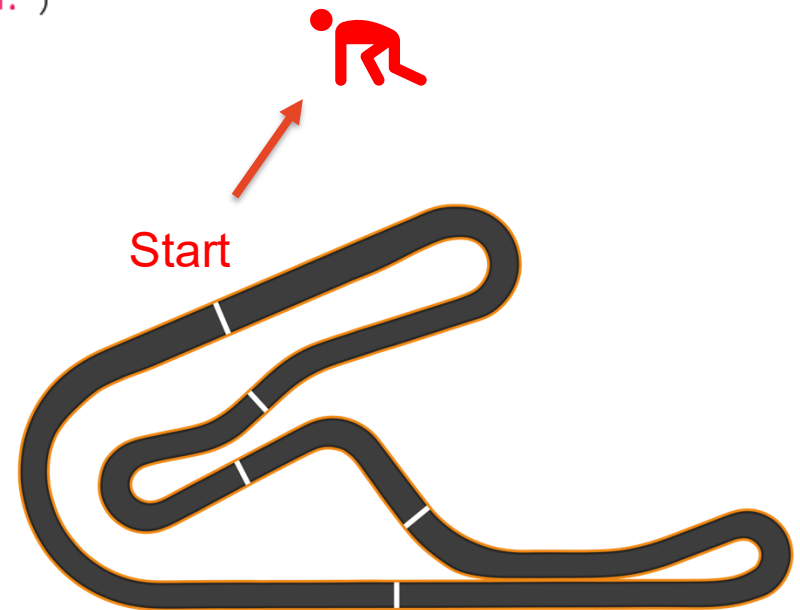
- Draw a flowchart of your actions.



Loops IRL: Ideal Case



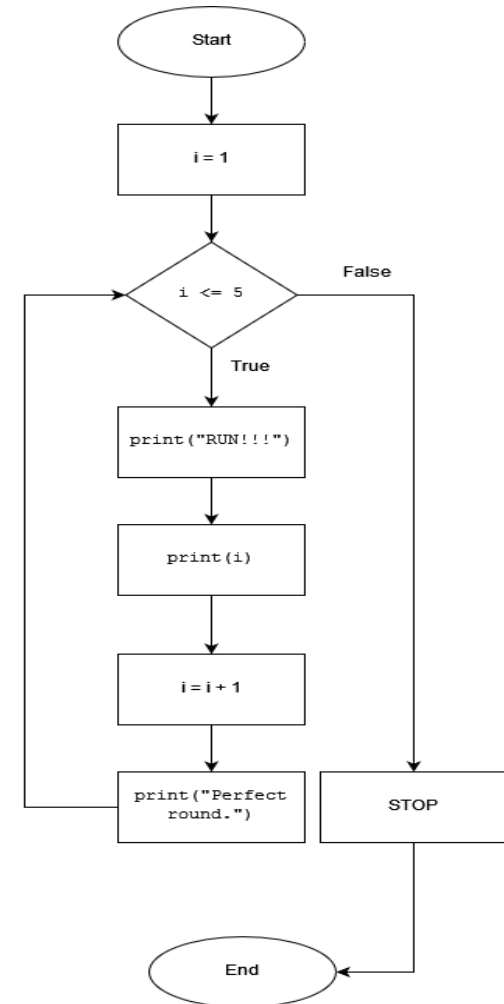
```
1 i = 1
2 while i<=5:
3     print("RUN!!!")
4     print(i)
5     i = i + 1
6     print("Perfect round.")
7 print("STOP")
8 # End program
```



Run [basic.py](#) to view the output.

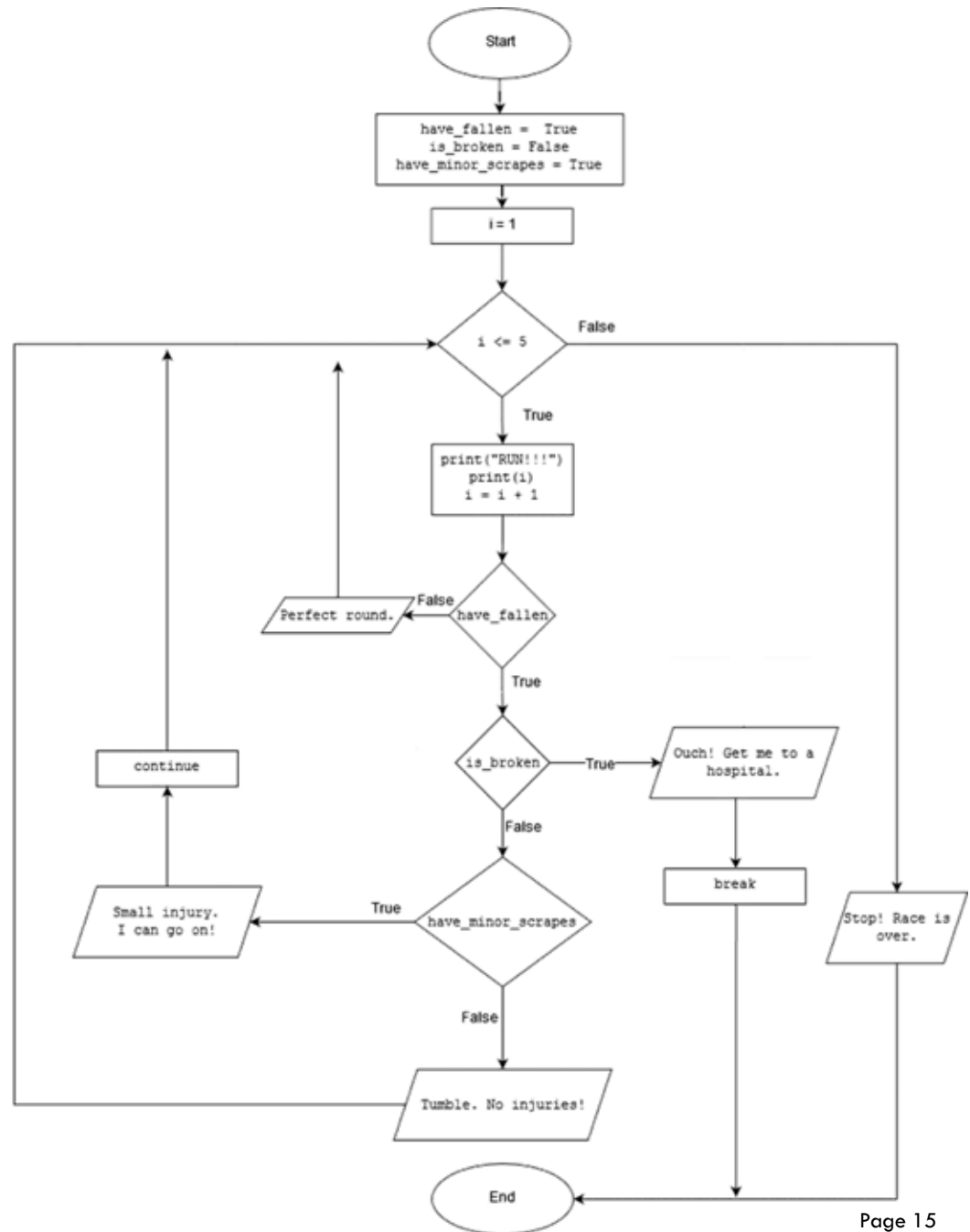
Loops IRL: When accidents happen...

- What would you do during the race if you:
 - Fell and scraped your knee?
 - Fell and broke your leg?
- Decision-making algorithm:
 - If you have fallen, assess the injury.
 - Broken leg, stop race.
 - Scrape knee, continue race.
 - No injuries only fall-prone, race as normal.
- Where should this “assess injury algorithm” be placed in the flowchart?



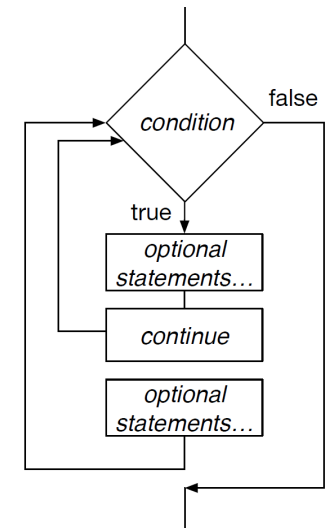
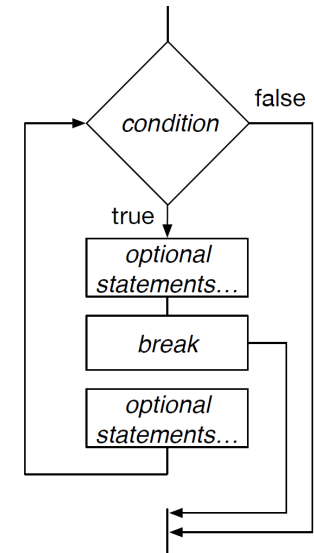
Your task:

- Trace the pathway to work out the output for every round during the race.
- How do you get a Perfect round. on all rounds?
- If you have fallen and broken your leg, what would be the program's output?
- Try: Race again: Break/continue.
- Race again: Random luck



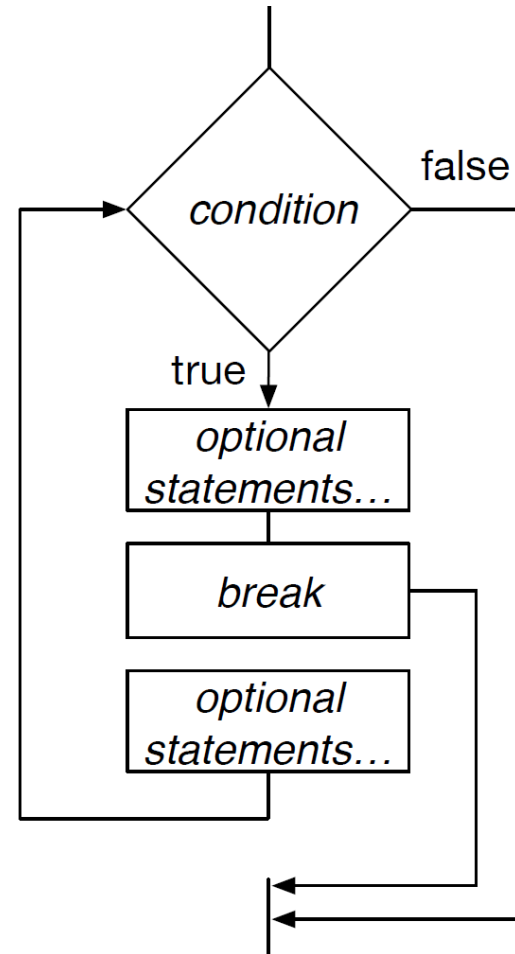
Nutshell: break and continue

Keywords	break	continue
Purpose	Immediate exit from the <u>current loop</u> . Remaining statements in the loop are ignored.	Skip the rest of this iteration and go on to the next one.
Syntax	break	continue
Where?	Body of loops	Body of loops

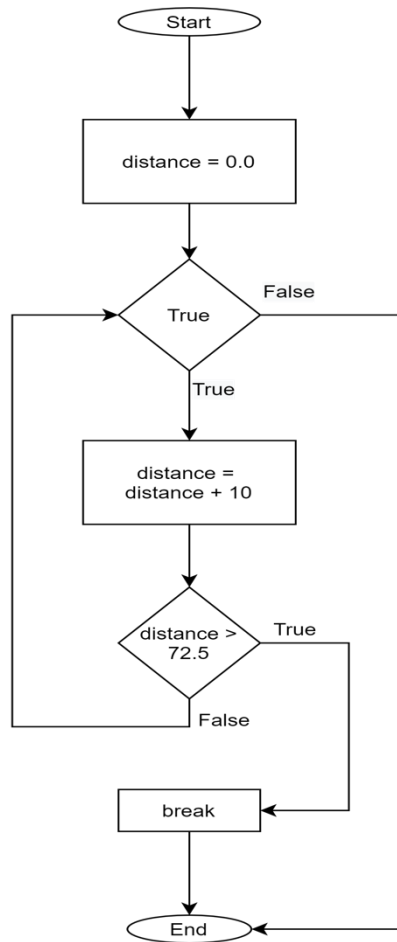


break

- Special reserved keyword.
- When used in loops, it means ‘break’ from the *current* iteration of this loop.
- It is only allowed *inside* the body of a loop.



Practice: break



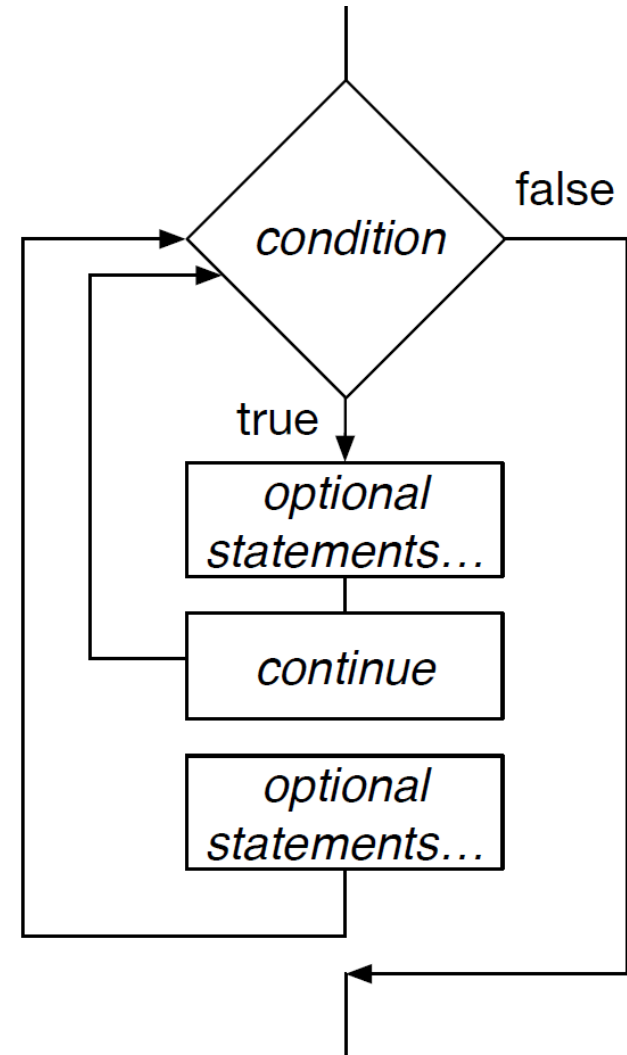
- What is the value of distance at the end of the program?

distance	distance+10	distance>72.5
0.0		

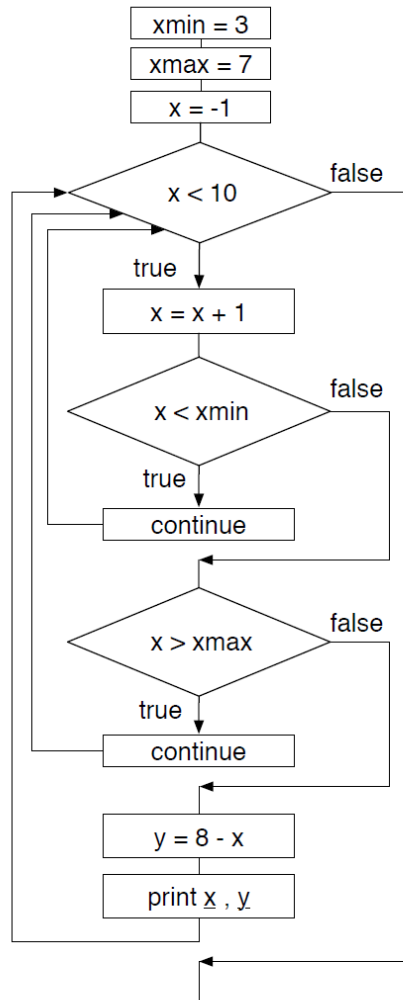
- Implement the flowchart: [Ed Lesson](#)

continue

- Another reserved keyword.
- When used in loops, it means ‘skip the rest of this iteration and go on to the next one’
- It is only allowed *inside* the body of a loop.



Practice: continue



- What are the values of `x` and `y` at the end of the program?

x	y
-1	

- Implement the flowchart: [Ed Lesson](#)

Practice: break and continue

- What are the values of variables `x` and `y` at the end of the program?
- Identify the output of the program.
- What would the program output if the increment of `x` is move just after line 5 before line 6?

```
1 x = 4
2 y = 20
3 while x < y:
4     x += 1
5     if x % 2 == 0:
6         continue
7     print("x = " + str(x))
8     if x == 13:
9         break
10 print("All done!")
11 # Author: Dr. John Stavrakakis
```

Trace in [Ed Lesson](#)

Practice: break and continue

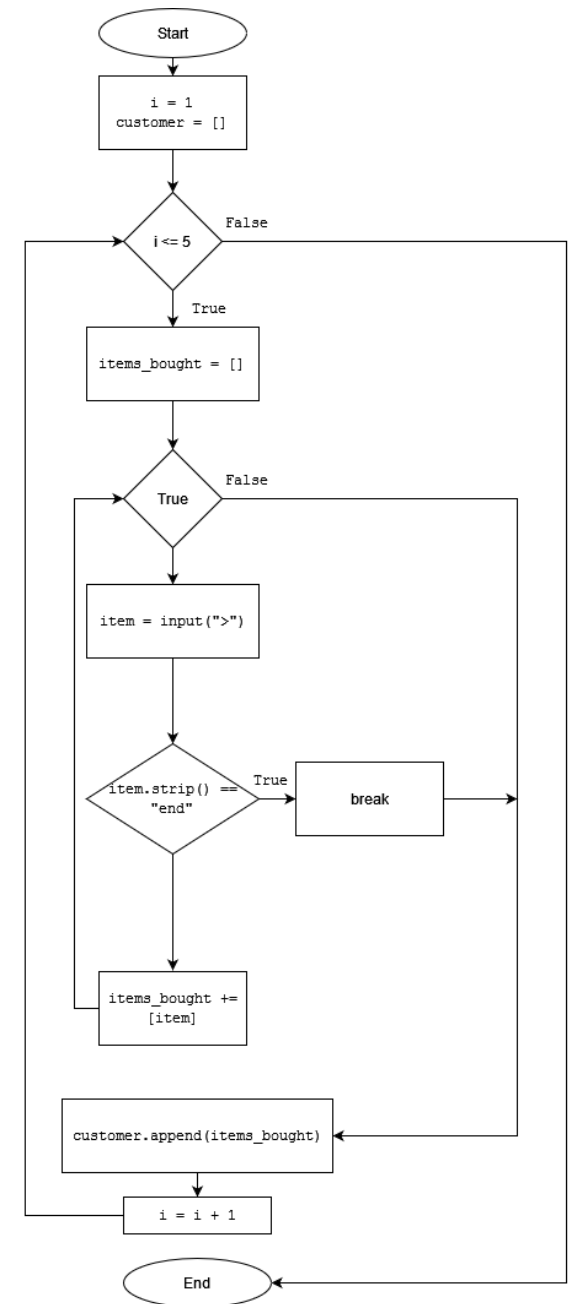
x	y	Output
4	20	
5	20	x = 5
6	20	
7	20	x = 7
8	20	
9	20	x = 9
10	20	
11	20	x = 11
12	20	
13	20	x = 13
13	20	All done!

```
1 x = 4
2 y = 20
3 while x < y:
4     x += 1
5     if x % 2 == 0:
6         continue
7     print("x = " + str(x))
8     if x == 13:
9         break
10 print("All done!")
11 # Author: Dr. John Stavrakakis
```

Trace in [Ed Lesson](#)

Practice: Nested loops with break

- Identify the count-controlled loop and event-controlled loop.
- Would the `break` statement exit the outer loop?
- What are the data types stored in these lists:
 - `items_bought`?
 - `customer`?
- Practice writing code on [Ed Lesson](#)



Menti Quiz Time!



<https://www.menti.com/aldecxh4xonp>

Break!

for loop

- Python for statement syntax a bit different from while loop:
 - No statements to specify terminating condition or iteration step
 - an **iterator-based loop**, which steps through the items of iterable objects like lists, tuples, string and executes a piece of code repeatedly for a number of times, based on the number of items in that iterable object.
- Syntax:

```
for <variable> in <collection>:  
    code block
```

1. <variable> initialise with the first item of the collection.
 2. Execute everything in code block
 3. Assign variable to the next element in the collection. Do Step 2.
- [Go to Ed Lesson](#) and do Part 1.

Revisit: `range(start, stop, step)`

- Way to construct a sequence of numbers.
- Commonly used for looping a specific number of times in for loops.
- Syntax:
 - `range(stop)`
 - creates sequence from 0 to integer specified by `stop-1`
 - `range(3)` → sequence: 0, 1, 2
 - `range(start, stop, step)`
 - creates sequence from `start` to integer specified by `stop-1`
 - If `step` is unspecified, by default is 1
 - `range(1, 4)` → sequence: 1, 2, 3
 - `range(1, 10, 2)` → sequence: 1, 3, 5, 7, 9
 - `range(10, 1, -2)` → sequence: 10, 8, 6, 4, 2
 - `range(10, 1)` → sequence:

<https://docs.python.org/3/library/stdtypes.html#typeseq-range>

Revisit: From while loop to for loop

- Now, can you spot the similarities between both loops?

```
1 i = 1
2 while i <= 5:
3     print(f"Round {i}: RUN!!!")
4     i = i + 1
5 print(f"Round {i}: Stop!")
```

```
1 for i in range(1, 6):
2     print(f"Round {i}: RUN!!!")
3 print(f"Round {i+1}: Stop!")
```

- Can you identify these in the `for` loop?
 - Loop control variable
 - Terminating condition
 - Modification of control variable

Revisit: return

Revisit: Functions _ What we know so far from W6?

- Functions allows you to refer to a set of instructions by its (function) name. There are four parts to each function:
 - The function **name** (what it's called)
 - The function **arguments** (the information/variables we pass it).
 - The **return** type (what kind of thing is returned by the function).
 - The function **body** (the actual code that does the work)
- Code inside the function body will only execute during a function call.
- Can have none, one or many arguments. We can also have keyword arguments e.g. `sep` and `end` in `print()` function.
- Function scope matters. Consequences:
 - Local variables inside the functions cannot be used outside the function unless returned as a return-value.
 - Global variables can be seen and used inside **Python** function but you cannot modify them.

Revisit: Functions _ What we know so far from W6?

- Strings and tuples are immutable. Lists are mutable.
- We can pass lists and tuples as arguments into a function.
- Two ways of passing arguments into the function:
 - Pass by value:
 - A copy of the argument is passed to the function.
 - Modification to the value inside function does not affect original copy.
 - Pass by reference:
 - Memory address of the argument is passed to the function.
 - Modification to the value inside the function affects original copy.
- We can return a `list` out of functions, but `tuples` are preferred to prevent modification of the result!

Revisit: return

- Review the code given in [Ed Lesson](#).
- What is the output of the program?

```
1 def my_func():
2     i = 1
3     while i <= 5:
4         j = 1
5         while j <= 5:
6             if i == 2 and j == 3:
7                 break
8             j += 1
9             i += 1
10
11 def my_func2():
12     i = 1
13     while i <= 5:
14         j = 1
15         while j <= 5:
16             if i == 2 and j == 3:
17                 return "Done"
18             j += 1
19             i += 1
20     return "End"
21
22 i = 50
23 j = 20
24 print(my_func())
25 print(my_func2())
26 print(i, j)
27 # End program
```

Revisit: `return` Observations

- The `return` statement will always return something (type `None`) even if there is no explicit `return` statement.
- You can only return ONE data type.
 - `my_func()` returns `None`
 - `my_func2()` returns `str`
- `return` statements return a value and terminates the execution of the function. Hence, code inside the function body after a `return` statement will not execute.
- **There can be multiple `return` statements in a function.**

return Multiple Values

- Review the codes in [demo.py](#).
 - What is the data type of the return value?

```
1 def add_diff(x, y):
2     '''
3     Function to find the addition and subtraction of two numbers.
4     Parameters:
5         x, y are numeric and positive values.
6     Return:
7         A tuple of the result of calculation: (add, diff)
8         add: Addition of x and y
9         diff: Difference between x and y
10        None if calculation cannot be performed
11    '''
12    add = x + y
13    diff = x - y
14    return add, diff
```

Premature return

- In your opinion, should the function execute if the argument types are not int or float?
- Modify the codes to check if `x` and `y` are of type `int` or `float`. If they are not, return `None`.
 - How do you test the type of an object?
 - Recall L2, everything in Python is an object. `type()` function returns the class of the object.
 - Another function to check whether an object is an instance of a given class, `isinstance()`. Run codes in [Part 2](#).
 - Where should you put the codes for checking?
 - Type checking should be done at the start of the function before any actions happen!

Try

Exception Handling

Raise



raise statement

- Wait, why don't we stop execution completely and inform the programmer that something is wrong?
 - raise keyword : Manually raise an exception
- ```
raise <exception_type>(<msg>)
```
- <exception\_type> can be specific from concrete Exception or general Exception
    - Error message on terminal: <exception\_type>: <msg>
  - Ed Lesson. Un-comment the function call one at a time in `demo.py` and run the program. Identify the line number that raises:
    - A general exception?
    - A `ValueError` exception?

# Handling Exceptions: `try...except`

- What if we don't want the program to stop executing but display a useful error message?
- We can make our programs handle exceptions gracefully using `try` and `except`.

- **Syntax:**

```
try:
 <statements we intend to do>
except <ExceptionName>:
 <statements for handling errors>
```

- Review `try_demo.py` in [Ed Lesson](#).

# Handling Exceptions: `try...except`

- Put codes where you suspect an error may occur into the `try` block.
- The `except` block will only execute when an exception occurs.
- The general `except` block must be placed at the bottom.

```
1 try:
2 # do something
3 # which could throw
4 # Exceptions
5 except EOFError:
6 # do something to deal
7 # with the situation
8 except FileNotFoundError:
9 # do something else
10 except Exception:
11 # do something general
12 finally:
13 # code that should happen
14 # after normal/weird case
```

# Summary: break, continue, return, raise

| Keywords | break                                 | continue                                                                         | return                                                                                            | raise                                                                   |
|----------|---------------------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Purpose  | Immediate exit from the <b>loop</b> . | Skip the current iteration of the <b>loop</b> , and go on to the next iteration. | Return a value and terminate <b>function</b> . Codes below the return statement will not execute. | Raise an exception to stop execution in extraordinary(rare) situations. |
| Syntax   | <code>break</code>                    | <code>continue</code>                                                            | <code>return</code><br><code>&lt;return_value&gt;</code>                                          | <code>raise</code><br><code>&lt;exception_type&gt;(<br/>msg)</code>     |
| Where?   | Body of loops                         | Body of loops                                                                    | Body of functions and methods.                                                                    | Special occasions – when exceptional events happen.                     |

## Summary – Exception Handling

- `try` a piece of code that contains potential rare error
- If the rare error occurs a new exception is generated. `raise`
- The exception is caught using `except` and handled
- Following the `try`, error or not, more code can be executed with a block `finally`.



# Reading This Week

- Python documentation:
  - [https://docs.python.org/3/reference/simple\\_stmts.html#the-break-statement](https://docs.python.org/3/reference/simple_stmts.html#the-break-statement)
  - [https://docs.python.org/3/reference/simple\\_stmts.html#the-continue-statement](https://docs.python.org/3/reference/simple_stmts.html#the-continue-statement)
- **Chapter 7. break.** Downey, A. B. (2015). Think Python: How to Think Like a Computer Scientist (2e ed.). O'Reilly Media, Incorporated.

# Reading This Week

- **Chapter 12. Tuples as return values.** Downey, A. B. (2015). Think Python: How to Think Like a Computer Scientist (2e ed.). O'Reilly Media, Incorporated.
- **Chapter 2.1.** Passing arguments and returning values. Sedgewick, R., Wayne, K., & Dondero, R. (2015). Introduction to programming in Python: An interdisciplinary approach. Addison-Wesley Professional.
- **Chapter 7. break.** Downey, A. B. (2015). Think Python: How to Think Like a Computer Scientist (2e ed.). O'Reilly Media, Incorporated.
- **Chapter 5.10 (break and continue)** Liang, Y. (2022). Introduction to Python Programming and Data Structures, Global Edition. Pearson Education Limited.
- **Chapter 6.10 (multiple return values)** Liang, Y. (2022). Introduction to Python Programming and Data Structures, Global Edition. Pearson Education Limited.
- **Chapter 13.7 (Raising Exceptions)** Liang, Y. (2022). Introduction to Python Programming and Data Structures, Global Edition. Pearson Education Limited.
- **Chapter 5.6. For statements.** Liang, Y. (2022). Introduction to Python Programming and Data Structures, Global Edition. Pearson Education Limited.
- **Python documents:**
  - [https://docs.python.org/3/reference/simple\\_stmts.html#the-assert-statement](https://docs.python.org/3/reference/simple_stmts.html#the-assert-statement)
  - [https://docs.python.org/3/reference/simple\\_stmts.html#the-raise-statement](https://docs.python.org/3/reference/simple_stmts.html#the-raise-statement)

Fully anonymous feedback form  
for COMP9001 students



Fully anonymous feedback form for COMP9001 students – Fill  
out form

# DISCOVERY ACTIVITY

You are doing late-night shopping in Westfields and have a budget of \$100. There are 6 stores left open. Each of them sells items between \$10 and \$15.

- (1) Write a program that visits all 6 stores and purchases the most expensive item, then prints the remaining budget at the end.
- (2) When you visit the 3<sup>rd</sup> store, you don't see anything you like, so you don't purchase anything. Update your code to reflect this.
- (3) After visiting the 5<sup>th</sup> store, there is a centre announcement that all shops are closing. Update your code to reflect this.
- (4) You saw something you wanted to buy at the 4<sup>th</sup> store, but there was no price tag. Raise this issue with customer service.
- (5) Due to the closing hours, the stores reduce the prices to randomly be between \$8-\$12. Update your code to reflect this.
- (6) You revisit the centre the next day but realise you have spent all your money. So you try to pay using "coupons", but the stores do not accept a "coupon". Write a try-except block to allow you to safely exit the centre.
- (7) Finally, print "Shopping trip complete with budget X" at the end of your code.

